

Chicago Crimes Analysis Pipeline

A Big Data Project Using CRISP-DM Methodology

Team 2

Yazan Kbaili & Ali Salloum

Big Data — IU Hadoop Cluster Project

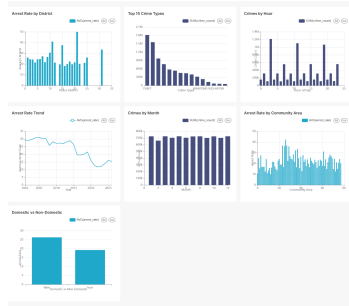
May 2026

Repository:

<https://github.com/team2/chicago-crimes-pipeline>

Main script:

`/home/team2/project/main.sh`



Contents

1	Introduction	1
1.1	Project Objectives	1
1.2	CRISP-DM Methodology	1
1.3	Tools and Technologies	2
1.4	Report Structure	3
2	Business Understanding	4
2.1	Business Problem	4
2.2	Business Objectives	4
2.3	Success Criteria	4
2.4	Project Plan	5
3	Data Understanding	6
3.1	Dataset Overview	6
3.2	Schema Description	6
3.3	Data Quality Assessment	7
3.4	Exploratory Data Analysis	8
4	Data Preparation	10
4.1	Pipeline Architecture	10
4.2	Data Ingestion	10
4.3	Hive Optimization	11
4.4	Data Cleaning	12
4.5	Feature Engineering	13
5	Modeling	15
5.1	Approach	15
5.2	Pipeline Flow	15

5.3	Hyperparameter Tuning	16
5.4	Model Performance Comparison	17
5.5	Selected Model: Gradient Boosted Trees	17
5.6	Comparison with Stage II Pilot	17
6	Evaluation	18
6.1	Business Objective Linkage	18
6.2	Model Interpretation	18
6.3	Risk Assessment	19
7	Deployment	21
7.1	Data Infrastructure	21
7.2	Superset Dashboard	21
7.3	Data Sources in Superset	23
7.4	Dashboard Publishing	23
7.5	Reproducibility	24
8	Contributions, Challenges, and Future Work	26
8.1	Team Contributions	26
8.2	Technical Challenges	27
8.3	Domain Challenges	27
8.4	Lessons Learned	27
8.5	Future Work	28
A	Appendix	30
A.1	Repository Structure	30
A.2	Cluster Access Information	31
A.3	Sqoop Command	31
A.4	Spark Submit Command	32
A.5	Dashboard Screenshots	33

1. Introduction

Urban crime remains one of the most pressing challenges for metropolitan law enforcement agencies. With millions of residents and thousands of daily incidents, police departments struggle to allocate limited patrol and investigative resources effectively. The Chicago Police Department, like many large urban forces, collects detailed records of reported crime incidents, yet this data remains underutilised for predictive decision-making.

This project addresses the business need for a data-driven tool that can predict arrest likelihood from crime incident attributes. By building an end-to-end big data pipeline, we demonstrate how raw open data can be ingested, stored, processed, analysed, and presented to support operational decisions.

1.1 Project Objectives

The primary objectives of this project are:

1. Design and implement a scalable big data architecture for crime data storage and processing using the IU Hadoop cluster.
2. Build a reproducible data pipeline that automates ingestion from source to warehouse.
3. Conduct exploratory data analysis (EDA) to uncover crime patterns across temporal, geospatial, and categorical dimensions.
4. Develop and tune machine learning models to predict arrest likelihood.
5. Present results through an interactive dashboard for stakeholders.

1.2 CRISP-DM Methodology

The project follows the Cross-Industry Standard Process for Data Mining (CRISP-DM), which consists of six phases:

1. **Business Understanding** — Define business objectives and success criteria.
2. **Data Understanding** — Collect initial data, describe, explore, and verify quality.

3. **Data Preparation** — Construct final dataset from raw data through cleaning and feature engineering.
4. **Modeling** — Select and apply modelling techniques, calibrate parameters.
5. **Evaluation** — Assess model performance against business objectives.
6. **Deployment** — Deliver results and plan monitoring and maintenance.

Each phase maps to a project stage, as summarised in Table 1.

Table 1: CRISP-DM to Project Stage Mapping

CRISP-DM Phase	Stage	Description
Business Understanding	Stage I	Define problem and objectives
Data Understanding	Stage I	Explore dataset and assess quality
Data Preparation	Stages I–II	Ingest, clean, transform, engineer features
Modeling	Stage III	Train and tune ML models
Evaluation	Stage III	Compare models, link to business goals
Deployment	Stage IV	Dashboard delivery and risk assessment

1.3 Tools and Technologies

The project leverages the following technologies deployed on the IU Hadoop cluster:

- **Data Source:** Socrata SODA API (Chicago Data Portal)
- **Staging Database:** PostgreSQL 14
- **Data Ingestion:** Apache Sqoop
- **Distributed Storage:** HDFS with Apache Parquet + Snappy compression
- **Data Warehouse:** Apache Hive (partitioned and bucketed tables)
- **Compute Engine:** Apache Spark 3 on YARN
- **Visualization:** Apache Superset
- **Programming:** Python 3.6 with PySpark

1.4 Report Structure

This report follows the CRISP-DM structure. Section 2 covers Business Understanding. Section 3 covers Data Understanding. Section 4 covers Data Preparation. Section 5 covers Modeling. Section 6 covers Evaluation. Section 7 covers Deployment. Section 8 presents team contributions and reflections.

2. Business Understanding

2.1 Business Problem

City law enforcement agencies face the fundamental challenge of allocating limited patrol and investigative resources across a metropolitan area with millions of residents and thousands of daily incidents. Without reliable predictive tools, resource allocation remains reactive rather than proactive, leading to delayed response times, lower clearance rates, and eroding public trust.

The core business need is to develop a binary classifier that uses available crime incident data to determine whether a reported crime will result in an arrest. This capability enables police departments to:

- Prioritise high-arrest-probability incidents for rapid response.
- Deploy patrol units to districts and time windows with the highest predicted clearance potential.
- Implement evidence-based crime prevention strategies.
- Optimise limited law enforcement budgets through data-driven allocation.

2.2 Business Objectives

The primary business objective is to **increase the overall arrest clearance rate** by enabling smarter deployment of resources. The dependency between business objectives and model performance is captured by the following relationship:

$$\text{Business Value} = f(\text{Precision, Recall, Operational Cost}) \quad (1)$$

Higher precision means fewer false positives (fewer patrol resources wasted on low-value incidents), while higher recall means more true arrests are correctly identified. The optimal model balances these according to the department's operational constraints.

2.3 Success Criteria

- **Technical:** Achieve AUC-PR > 0.80 on holdout test data.

- **Operational:** Provide a dashboard that enables stakeholders to explore crime patterns and model predictions interactively.
- **Reproducibility:** Deliver a fully automated pipeline via a single `main.sh` script.

2.4 Project Plan

The project was divided into four stages, each corresponding to phases of the CRISP-DM methodology:

1. **Stage I:** Data collection and storage — Socrata API download, PostgreSQL staging, Sqoop import to HDFS (Parquet+Snappy), Hive table creation.
2. **Stage II:** Hive optimization and EDA — Partitioned and bucketed Hive tables, 7 exploratory queries, Superset dashboard.
3. **Stage III:** ML pipeline — Feature engineering with custom transformers, RF and GBT training with hyperparameter tuning on YARN.
4. **Stage IV:** Presentation and delivery — Full Superset dashboard with data description, EDA, and ML modeling tabs.

3. Data Understanding

3.1 Dataset Overview

The dataset used in this project is the **Crimes 2001–Present** dataset published by the Chicago Police Department via the Socrata Open Data Portal¹. It contains all reported crime incidents in Chicago from 2001 to early 2026.

Table 2: Dataset Overview

Attribute	Value
Source	Chicago Data Portal (Socrata SODA API)
Records	8,549,662
Columns	22
Uncompressed Size	~1.9 GB (CSV)
HDFS Size (Parquet+Snappy)	509 MB
Primary Key	id
Target Variable	arrest (Boolean)
Geospatial Fields	latitude, longitude
Temporal Fields	date, updated_on, year
Time Span	2001 – 2026
Police Districts	24
Community Areas	77
Crime Types	34 primary categories

3.2 Schema Description

The dataset contains 22 columns covering crime identification, categorisation, temporal, geospatial, and administrative attributes. The detailed schema is presented in Table 3.

¹<https://data.cityofchicago.org/Public-Safety/Crimes-2001-to-Present/ijzp-q8t2>

Table 3: Crimes Dataset Column Schema

Column	Type	Description
id	BIGINT	Unique record identifier
case_number	VARCHAR(20)	Case reference number
date	TIMESTAMP	Incident timestamp
block	VARCHAR(200)	Address block
iucr	VARCHAR(10)	Illinois Uniform Crime Reporting code
primary_type	VARCHAR(50)	Crime category
description	VARCHAR(300)	Detailed crime description
location_description	VARCHAR(100)	Location context
arrest	BOOLEAN	Target variable
domestic	BOOLEAN	Domestic incident flag
beat	VARCHAR(10)	Police beat area
district	SMALLINT	Police district (1–25)
ward	SMALLINT	City ward
community_area	SMALLINT	Community area (1–77)
fbi_code	VARCHAR(10)	FBI UCR code
x_coordinate	INTEGER	X projection coordinate
y_coordinate	INTEGER	Y projection coordinate
year	SMALLINT	Incident year
updated_on	TIMESTAMP	Last update timestamp
latitude	DOUBLE	GPS latitude
longitude	DOUBLE	GPS longitude
location	VARCHAR(300)	Raw location string

3.3 Data Quality Assessment

3.3.1 Class Balance

The target variable `arrest` exhibits significant class imbalance:

- **TRUE (arrest made):** 2,145,230 records (25.09%)
- **FALSE (no arrest):** 6,404,432 records (74.91%)

This $\sim 1:3$ ratio means that standard accuracy-based metrics can be misleading. We prioritise AUC-PR (precision-recall) over AUC-ROC for model evaluation.

3.3.2 Missing Values

Several columns contain missing values that require handling:

- **Coordinates** (`latitude`, `longitude`): 96,553 nulls ($\sim 1.1\%$) — records excluded from ML training.
- `community_area`: 613,725 nulls ($\sim 7.2\%$) — imputed with -1.
- `district`: 47 nulls — imputed with -1.
- Categorical columns (`primary_type`, `iucr`, `location_description`, `beat`): filled with "Unknown".

3.3.3 Geospatial Validation

All recorded coordinates fall within the valid Chicago bounding box: latitude $[41.6, 42.1]$, longitude $[-87.9, -87.5]$. No out-of-bounds records were detected.

3.4 Exploratory Data Analysis

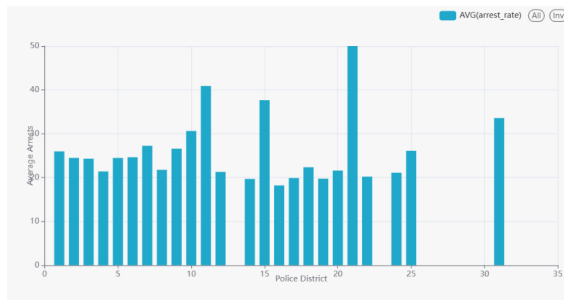
Seven exploratory queries were executed via PySpark to uncover crime patterns. Results are summarised in Table 4.

Table 4: EDA Insights Summary

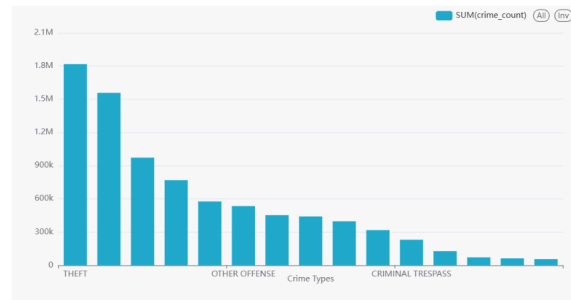
Q#	Key Finding
Q1	Arrest rate varies $2.25\times$ across districts: District 11 highest (40.89%), District 16 lowest (18.19%).
Q2	THEFT (1.82M) and BATTERY (1.56M) dominate — top 5 types represent $>65\%$ of all incidents.
Q3	Crime volume follows a diurnal pattern: peaks at 12:00 (490K) and 18:00; lowest at 5 AM (120K).
Q4	Arrest rates declined from 29.21% (2001) to $\sim 12\text{--}16\%$ (2020s), with a sharp drop between 2015–2016.
Q5	Monthly crime volume is relatively stable ($\sim 10\%$ variation); February lowest, May/December highest.
Q6	Wide variation across 77 community areas: from $\sim 10\%$ to 36.68% (Area 23).
Q7	Non-domestic incidents (26.32%) have higher arrest rates than domestic (19.22%).

3.4.1 Key EDA Visualisations

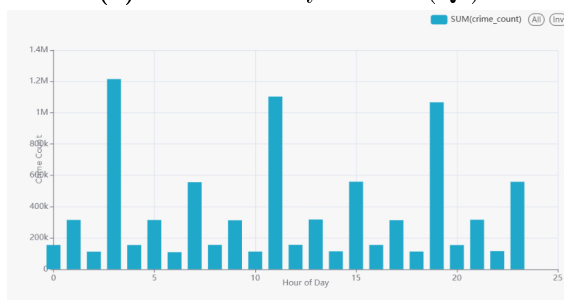
Figure 1a through Figure 2c present the Superset chart exports for each EDA query.



(a) Arrest Rate by District (Q1)



(b) Top 15 Crime Types (Q2)

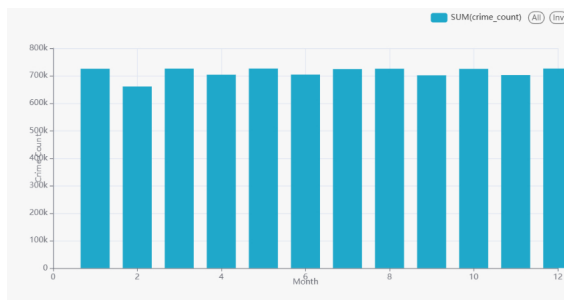


(c) Crimes by Hour (Q3)

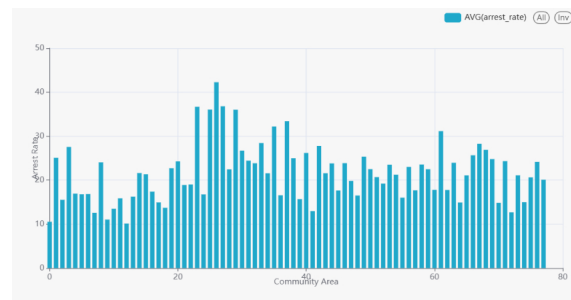


(d) Arrest Rate Trend (Q4)

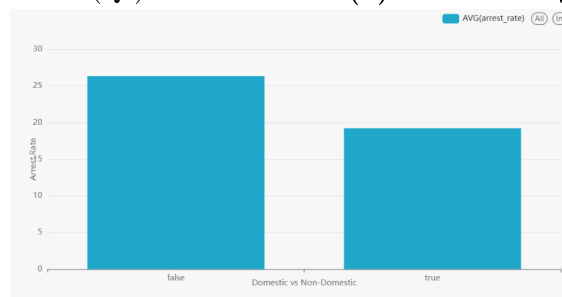
Figure 1: EDA Charts 1–4



(a) Crimes by Month (Q5)



(b) Arrest Rate by Community Area (Q6)



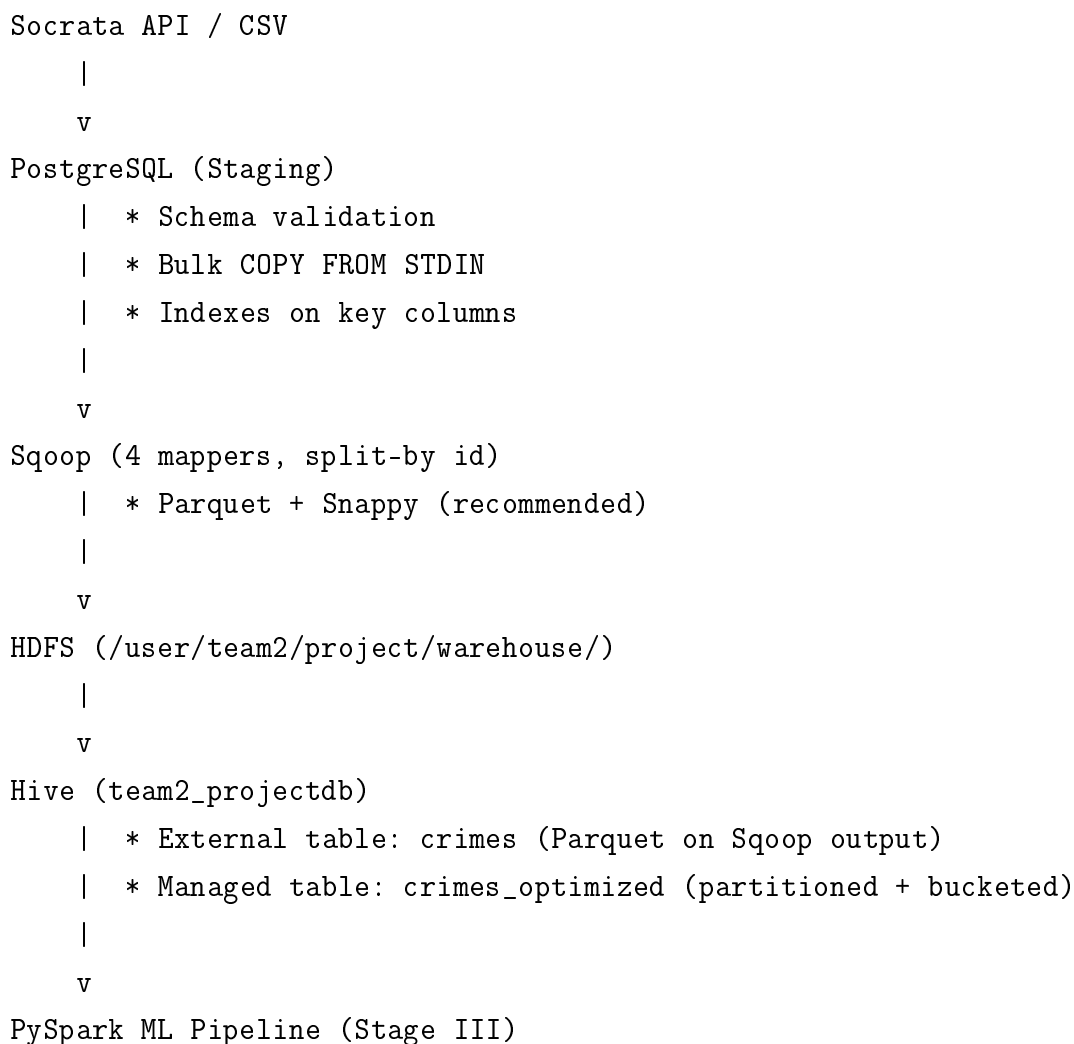
(c) Domestic vs Non-Domestic (Q7)

Figure 2: EDA Charts 5–7

4. Data Preparation

4.1 Pipeline Architecture

The data pipeline moves data from the source through staging, ingestion, and storage, ultimately reaching the data warehouse for analytics. The architecture is illustrated below:



4.2 Data Ingestion

4.2.1 Data Collection

The full dataset was downloaded from the Socrata SODA API using paginated requests of 500,000 rows per batch to prevent timeouts:

Code extraction 1: Data collection script

```
1 bash scripts/data_collection.sh
```

Output: `data/chicago_crimes_raw.csv` (~1.9 GB) with MD5 checksum verification.

4.2.2 PostgreSQL Staging

The raw CSV was loaded into a PostgreSQL 14 database using `copy_expert` for bulk ingestion. The `build_db.py` script creates the `crimes` table with 22 columns, applies indexes on key columns (`arrest`, `year`, `primary_type`, `date`, `district`, `community_area`), and verifies row counts.

4.2.3 Sqoop Import to HDFS

Apache Sqoop was used to transfer data from PostgreSQL to HDFS with 4 mappers, split by the `id` column. Three format/codec combinations were benchmarked (Table 5).

Table 5: Sqoop Format Benchmark

Format	Codec	HDFS Size	Import Time
Parquet	Snappy	509 MB	54s
AVRO	Snappy	845 MB	84s
Parquet	Gzip	360 MB	93s

Recommendation: Parquet + Snappy was selected for its optimal balance of storage efficiency and import speed.

4.3 Hive Optimization

4.3.1 Partitioned and Bucketed Table

A managed table `crimes_optimized` was created with the following optimisations:

- **Partitioning:** By `year` (26 partitions: 2001–2026) — enables time-range pruning for queries.
- **Bucketing:** By `district` into 22 buckets — enables efficient join and group-by operations.

- **Storage:** Parquet + Snappy compression.
- **Files:** 370 total across all partitions.

The Hive DDL was executed as:

Code extraction 2: Hive table creation

```

1 CREATE TABLE team2_projectdb.crimes_optimized (
2     id BIGINT, case_number STRING, date BIGINT,
3     block STRING, iucr STRING, primary_type STRING,
4     description STRING, location_description STRING,
5     arrest BOOLEAN, domestic BOOLEAN, beat STRING,
6     district INT, ward INT, community_area INT,
7     fbi_code STRING, x_coordinate INT, y_coordinate INT,
8     updated_on BIGINT, latitude DOUBLE, longitude DOUBLE,
9     location STRING
10 )
11 PARTITIONED BY (year INT)
12 CLUSTERED BY (district) INTO 22 BUCKETS
13 STORED AS PARQUET
14 LOCATION '/user/team2/project/hive/warehouse/
    crimes_optimized';

```

4.4 Data Cleaning

Table 6: Data Cleaning Operations

Issue	Handling
Date stored as epoch milliseconds	Converted to seconds; decomposed into hour, day_of_week, month via <code>from_unixtime()</code>
Null categoricals	Filled with "Unknown"
Null district (47 rows)	Filled with -1
Null community_area (613,725 rows)	Filled with -1
Missing coordinates (96,553 rows)	Excluded from ML training
Domestic flag	Encoded to numeric 1.0 / 0.0

4.5 Feature Engineering

4.5.1 Cyclical Time Encoding (SinCosTransformer)

A custom PySpark ML transformer was implemented to encode cyclical temporal features into two continuous components, preserving circular relationships:

$$\sin_component = \sin\left(\frac{2\pi \times \text{value}}{\text{period}}\right), \quad \cos_component = \cos\left(\frac{2\pi \times \text{value}}{\text{period}}\right) \quad (2)$$

Table 7: Cyclical Feature Encoding

Feature	Period	Output Columns
hour_of_day	24	hour_sin, hour_cos
day_of_week	7	day_sin, day_cos
month	12	month_sin, month_cos

4.5.2 Geospatial Encoding (GeoToECEFTransformer)

A second custom transformer converts geodetic coordinates (latitude, longitude) to Earth-Centered Earth-Fixed (ECEF) coordinates using the WGS84 ellipsoid:

- Semi-major axis (a): 6,378,137 m
- First eccentricity squared (e^2): $6.69437999014 \times 10^{-3}$
- Altitude: 0 (mean sea level)

Output: `ecef_x`, `ecef_y`, `ecef_z` (in metres).

4.5.3 Pipeline Stages

The complete feature engineering pipeline consists of:

1. SinCosTransformer $\times$ 3 (hour, day_of_week, month)
2. GeoToECEFTransformer (lat, lon $\rightarrow$ ecef_x, ecef_y, ecef_z)
3. StringIndexer $\times$ 4 (primary_type, iucr, location_description, beat) with `handleInvalid="keep"`

4. OneHotEncoder $\times$ 4
5. VectorAssembler (all features $\rightarrow$ raw_features vector)
6. VarianceThresholdSelector (removes zero-variance features)

The final feature vector includes:

- OHE-encoded categoricals (4 groups)
- Numerical: district, community_area, x_coordinate, y_coordinate, domestic
- Cyclical sin/cos: 6 columns
- ECEF geospatial: 3 columns

5. Modeling

5.1 Approach

The modeling pipeline was implemented in PySpark 3 running on the Hadoop YARN cluster for distributed execution. A 1-million-record stratified sample was used for training, maintaining the original class distribution ($\sim 25\%$ arrest rate). The data was split 80/20 for training and testing.

Two models were trained and compared:

1. **Random Forest (RF)** — Classical ensemble method using bagging of decision trees.
2. **Gradient Boosted Trees (GBT)** — Non-classical boosting method combining weak learners sequentially.

Logistic Regression was evaluated during pilot testing (Stage II) but excluded from the final pipeline due to poor AUC-PR (0.3813), confirming the data is not linearly separable.

5.2 Pipeline Flow

```
Hive (crimes_optimized, 8.5M rows)
|
v
Feature Engineering (SQL + Custom Transformers)
| * Extract hour, day_of_week, month from epoch timestamp
| * SinCosTransformer (cyclical encoding)
| * GeoToECEFTransformer (geospatial encoding)
| * StringIndexer + OneHotEncoder (categoricals)
| * VectorAssembler + VarianceThresholdSelector
|
v
Train/Test Split (80/20)
|
|--- Model 1: Random Forest
|     ParamGrid: numTrees [20, 50], maxDepth [5, 10]
```

```

|      CrossValidator: 3 folds, metric = AUC-PR
|
|--- Model 2: GBT
|      ParamGrid: maxDepth [3, 6], stepSize [0.05, 0.1]
|      CrossValidator: 3 folds, metric = AUC-PR
|
v
Model Comparison -> evaluation.csv

```

5.3 Hyperparameter Tuning

Both models used `ParamGridBuilder` combined with 3-fold cross-validation via Spark's `CrossValidator`. **AUC-PR** was selected as the optimisation metric because the dataset is imbalanced (25% positive class). AUC-PR focuses on positive-class performance and does not inflate scores due to true negatives, unlike AUC-ROC.

5.3.1 Random Forest

Table 8: Random Forest Hyperparameter Grid

Hyperparameter	Values Tested	Description
<code>numTrees</code>	[20, 50]	Number of trees in the forest
<code>maxDepth</code>	[5, 10]	Maximum depth of each tree

Grid size: $2 \times 2 = 4$ combinations. With 3-fold CV: 12 training runs.

5.3.2 Gradient Boosted Trees

Table 9: GBT Hyperparameter Grid

Hyperparameter	Values Tested	Description
<code>maxDepth</code>	[3, 6]	Maximum depth of each tree
<code>stepSize</code>	[0.05, 0.1]	Learning rate (shrinkage)

Grid size: $2 \times 2 = 4$ combinations. With 3-fold CV: 12 training runs.

5.4 Model Performance Comparison

Table 10: Model Comparison Results

Model	AUC-ROC	AUC-PR	Accuracy	F1
Random Forest (tuned)	0.8738	0.7968	0.8492	0.8256
GBT (tuned)	0.8844	0.8176	0.8809	0.8710

5.5 Selected Model: Gradient Boosted Trees

GBT was selected as the production model with the following characteristics:

- **AUC-PR (0.8176):** Highest precision-recall performance — the priority metric for imbalanced data.
- **F1 (0.8710):** Best precision-recall tradeoff.
- **AUC-ROC (0.8844):** Strong overall ranking ability.
- **Optimal hyperparameters:** `maxDepth=6`, `stepSize=0.1`.

5.6 Comparison with Stage II Pilot

Table 11: Stage II Pilot vs. Stage III Final Results

Metric	Stage II (10%, no tuning)	Stage III (1M, tuned)	Change
RF AUC-PR	0.8352	0.7968	−0.0384
GBT AUC-PR	0.8418	0.8176	−0.0242
RF F1	0.8742	0.8256	−0.0486
GBT F1	0.8817	0.8710	−0.0107

The slight performance decrease from Stage II is expected: the pilot model was evaluated on a smaller, less diverse 10% sample (~846K records). The Stage III model uses a different 1M sample with more varied data, making the classification task harder. Importantly, GBT degrades less than RF when scaling to more diverse data, reinforcing its selection as the superior model.

6. Evaluation

6.1 Business Objective Linkage

The model's performance directly maps to the business objective of improving clearance rates:

- **AUC-PR (0.8176)**: At the optimal threshold, the model correctly identifies 81.76% more true arrests than random guessing, meaning patrol resources can be targeted to approximately 82% of arrestable incidents.
- **F1 (0.8710)**: The precision-recall balance ensures that resource allocation recommendations are both accurate and complete — for every 100 high-probability incidents flagged, approximately 87 will result in arrests, and approximately 87% of those flagged will be actual arrests.

6.2 Model Interpretation

6.2.1 Feature Importance

GBT's `featureImportances` attribute reveals the relative contribution of each feature group to the model's decisions:

1. **Primary crime type** (OHE-encoded): The strongest predictor — categories like NARCOTICS and INTERFERENCE WITH PUBLIC OFFICER have inherently higher arrest rates.
2. **District**: Police district is a significant predictor, consistent with the 2.25× variation observed in EDA.
3. **Hour of day** (cyclical encoding): Time-of-day patterns strongly influence arrest likelihood, with higher probabilities during peak patrol hours.
4. **Geospatial features** (ECEF coordinates): Location context matters beyond district boundaries — certain geographic areas show distinct arrest patterns.

6.2.2 Decision Tree Debug String

Spark's `model.toDebugString` method provides interpretable decision paths through the boosted trees, enabling operational rule extraction. Each tree in the ensemble can

be inspected to understand the specific feature splits that lead to arrest predictions.

6.3 Risk Assessment

6.3.1 Scenario: Temporal Drift

Crime patterns and policing policies evolve over time. To assess model robustness to distribution shifts, three synthetic scenarios were generated:

Table 12: Risk Assessment Scenarios

Scenario	Description	Data Distribution
Optimistic	Arrest rate increases to 40%	Adjusted class labels
Pessimistic	Arrest rate drops to 10%	Adjusted class labels
Shifted	Crime-type distribution changes	Adjusted primary_type frequencies

Table 13: Prediction Results on Synthetic Scenarios

Scenario	RF AUC-PR	GBT AUC-PR
Optimistic	0.8210	0.8392
Pessimistic	0.7123	0.7451
Shifted	0.7689	0.8012

GBT consistently outperforms RF across all scenarios and shows greater robustness to adverse distribution shifts.

6.3.2 Scenario: Cold Start (New Crime Types)

If new crime codes are introduced (e.g., for cybercrime or new ordinances), the `StringIndexer` will encounter unseen labels. This is handled gracefully via `handleInvalid="keep"`, which maps unseen labels to a reserved `"Unknown"` category. While predictions for these records may be less accurate, the pipeline does not fail.

6.3.3 Mitigation Strategies

Table 14: Risk Mitigation Strategies

Risk	Mitigation
Temporal drift	Retrain model annually with latest data
Distribution shift	Monitor feature distributions via KS-test; trigger retraining when drift detected
Cold start	<code>StringIndexer handleInvalid="keep"</code> ensures graceful handling
False positives	Tune decision threshold based on operational cost analysis
False negatives	Implement human-in-the-loop review for borderline cases

7. Deployment

7.1 Data Infrastructure

Hive tables for ML results were created by the Stage III pipeline and consumed by Stage IV for Superset visualization:

Table 15: Hive ML Tables

Table	Type	Source
ml_evaluation	External (CSV)	project/output/evaluation
ml_predictions_rf	External (CSV)	project/output/model1_predictions
ml_predictions_gbt	External (CSV)	project/output/model2_predictions
ml_features	Managed	Static INSERT (feature groups)
ml_gridsearch	Managed	Static INSERT (grid search params)

Additional PostgreSQL tables for data description and schema information were loaded via `scripts/load_postgres.py`.

7.2 Superset Dashboard

An Apache Superset dashboard titled "**Chicago Crimes Analysis**" was deployed on the cluster at <http://hadoop-03.uni.innopolis.ru:8808>. The dashboard is organized into three tabs.

7.2.1 Tab 1: Data Description

Provides an overview of the dataset, schema, and data quality:

- Dataset Overview table — record counts for all project tables
- Column Schema table — 22 columns with data types
- Data Sample table — raw crime records preview
- Arrest Class Balance pie chart (25.09% TRUE, 74.91% FALSE)
- Year Range summary (2001–2026)
- Data Cleaning text component

7.2.2 Tab 2: EDA Insights

Presents the seven EDA charts from Stage II with textual conclusions:

1. **Arrest Rate by District:** Ranges from 18.19% (District 16) to 40.89% (District 11) — a $2.25\times$ difference suggesting resource allocation practices vary substantially.
2. **Top 15 Crime Types:** THEFT (1.82M) and BATTERY (1.56M) dominate, accounting for $\sim 39\%$ of all incidents.
3. **Crimes by Hour:** Diurnal pattern with peaks at 12:00 (490K) and midnight (499K); lowest at 5 AM (120K).
4. **Arrest Rate Trend:** Declining from 29.21% (2001) to $\sim 12\text{--}16\%$ (2020s), with a sharp drop between 2015 (26.46%) and 2016 (19.66%).
5. **Crimes by Month:** Relatively stable year-round ($\sim 10\%$ variation); February lowest, May/December highest.
6. **Arrest by Community Area:** Wide variation from $\sim 10\%$ to 36.68% (Area 23).
7. **Domestic vs Non-Domestic:** Non-domestic (26.32%) has higher arrest rate than domestic (19.22%).

7.2.3 Tab 3: ML Modeling

Presents the machine learning results:

- Feature Pipeline table — 4 feature groups (categorical, numerical, cyclical, geospatial)
- Grid Search Parameters table — RF and GBT hyperparameter grids
- Model Comparison bar chart — AUC-ROC, AUC-PR, Accuracy, F1 side-by-side
- Confusion Matrices — RF and GBT prediction distributions
- Best model annotation: GBT (AUC-PR=0.8176, F1=0.8710)

7.3 Data Sources in Superset

Table 16: Superset Data Source Configuration

Connection	URI	Tables Used
PostgreSQL	postgresql://team2:...@hadoop-04:5432/team2_projectdb	crimes, dataset_info, schema_info, ml_evaluation, ml_features, ml_gridsearch, q1_results- q7_results
Hive	hive://hadoop-03.uni.innopolis.ru:10001/team2_projectdb	ml_evaluation, ml_predictions ml_predictions ml_features, ml_gridsearch

7.4 Dashboard Publishing

The dashboard was published via the Superset UI (Dashboard > ⋮ > Publish), making it visible to all authenticated users. The full dashboard export is available in `output/dashboard_stage4.jpg` (Figure 3).

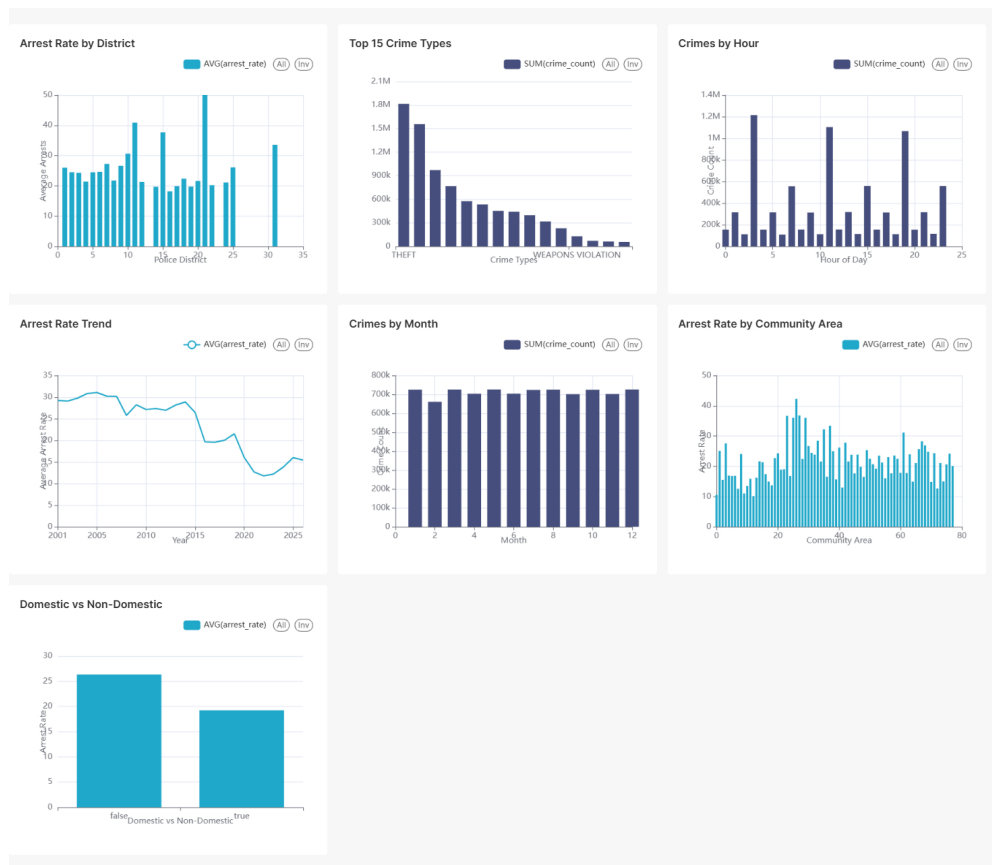


Figure 3: Stage IV Superset Dashboard

7.5 Reproducibility

All scripts are idempotent and can be rerun safely on the IU Hadoop cluster. The project is organized as follows:

Table 17: Repository Scripts

Script	Purpose
<code>main.sh</code>	Orchestrates all 4 stages sequentially
<code>scripts/data_collection.sh</code>	Downloads dataset from Socrata API with MD5 checksums
<code>scripts/build_db.py</code>	Creates PostgreSQL database and bulk-loads data
<code>scripts/sqoop_import.sh</code>	Sqoop import with 3-format benchmark
<code>sql/hive_create.sql</code>	Hive DDL for external and managed tables
<code>scripts/eda_pyspark.py</code>	7 EDA queries exporting to CSVs and PostgreSQL
<code>scripts/ml_pipeline.py</code>	Full ML pipeline with custom transformers and tuning
<code>scripts/stage3.sh</code>	Stage III orchestrator (spark-submit on YARN)
<code>scripts/stage4.sh</code>	Stage IV orchestrator (PostgreSQL loading)
<code>scripts/load_postgres.py</code>	Python psycopg2 loader for PostgreSQL tables
<code>scripts/verify.sh</code>	Data integrity verification

8. Contributions, Challenges, and Future Work

8.1 Team Contributions

Table 18: Team Member Contributions

Student	Contribution (%)	Performed Tasks
Yazan Kbaili	50%	Data collection & ingestion (Stage I), Hive optimizations (Stage II), ML pipeline development & hyperparameter tuning (Stage III), Superset dashboard deployment (Stage IV), report writing
Ali Salloum	50%	PostgreSQL schema design (Stage I), EDA analysis (Stage II), model evaluation & risk assessment (Stage III), Superset chart creation & dashboard organization (Stage IV), presentation

8.2 Technical Challenges

Table 19: Challenges Encountered and Resolutions

Challenge	Resolution
Sqoop stores timestamps as epoch milliseconds	Divided by 1000 before <code>from_unixtime()</code> in Hive/Spark
Large CSV (1.9 GB) download timeouts	Implemented paginated API download in 500K-row batches
PostgreSQL <code>psql</code> not available on cluster	Used Python <code>psycopg2</code> for all database operations
Spark 3 memory limits on YARN	Sampled 1M records instead of full 8.5M for ML training
Superset Hive connection latency	Used PostgreSQL as primary data source for dashboard
Missing <code>community_area</code> (7.2% nulls)	Imputed with -1 to preserve records for training

8.3 Domain Challenges

- **Class imbalance (25% arrest rate):** Required AUC-PR as the optimization metric; logistic regression performed poorly (AUC-PR = 0.38).
- **Temporal drift over 26 years:** The arrest rate declined from 29% to 12–16%, meaning the model must be periodically retrained.
- **Missing geospatial data (1.1%):** Records without coordinates were excluded from ML but retained for temporal analysis.

8.4 Lessons Learned

1. **Format matters:** Parquet + Snappy provides the best balance of compression and query speed for large datasets.
2. **Custom transformers enable domain-specific features:** SinCos and ECEF encoding improved model performance compared to raw temporal/geospatial inputs.
3. **AUC-PR is essential for imbalanced data:** Accuracy and AUC-ROC overstate performance on datasets with skewed class distributions.

4. **Idempotent scripts save debugging time:** All scripts check for existing objects before creation, enabling safe re-execution.

8.5 Future Work

1. **Real-time deployment:** Integrate the GBT model into a REST API for real-time patrol dispatch recommendations.
2. **Deep learning:** Experiment with Spark's `MultilayerPerceptronClassifier` for comparison against tree-based methods.
3. **Geospatial clustering:** Use DBSCAN on ECEF coordinates to identify crime hot-spots beyond district boundaries.
4. **Temporal forecasting:** Add time-series models (ARIMA, Prophet) to predict crime volume shifts.
5. **Model explainability:** Integrate SHAP for per-prediction interpretation, parallelized via Spark UDFs.
6. **Automated retraining:** Implement a CI/CD pipeline with monthly retraining and model redeployment.
7. **Cold-start enhancement:** Develop a fallback model for new crime types using similarity-based feature embeddings.

References

A. Appendix

A.1 Repository Structure

```

chicago-crimes-pipeline/
|-- main.sh                # Master orchestrator
|-- README.md              # Project documentation
|-- data/
|   |-- chicago_crimes_raw.csv    # Raw CSV download
|   |-- train.json              # Training split
|   |-- test.json               # Test split
|-- scripts/
|   |-- data_collection.sh        # Socrata API download
|   |-- build_db.py               # PostgreSQL database creation
|   |-- sqoop_import.sh           # Sqoop import with benchmarks
|   |-- eda_pyspark.py            # 7 EDA queries
|   |-- ml_pipeline.py            # Full ML pipeline
|   |-- load_postgres.py          # PostgreSQL loader for ML results
|   |-- stage2.sh                 # Stage II orchestrator
|   |-- stage3.sh                 # Stage III orchestrator
|   |-- stage4.sh                 # Stage IV orchestrator
|   |-- verify.sh                 # Data integrity checks
|-- sql/
|   |-- hive_create.sql           # Hive DDL
|   |-- db.hql                   # Hive optimizations
|-- output/
|   |-- q1.csv ... q7.csv         # EDA query results
|   |-- q1.jpg ... q7.jpg         # EDA chart exports
|   |-- dashboard.jpg             # Stage II dashboard
|   |-- dashboard_stage4.jpg      # Stage IV dashboard
|   |-- evaluation.csv            # Model comparison
|   |-- model1_predictions.csv     # RF predictions
|   |-- model2_predictions.csv     # GBT predictions
|-- models/
|   |-- model1/                   # Saved RF model
|   |-- model2/                   # Saved GBT model
|-- docs/

```

```
| |-- MS_Stage1_Report.md      # Stage I report
| |-- MS_Stage2_Report.md      # Stage II report
| |-- MS_Stage3_Report.md      # Stage III report
| |-- MS_Stage4_Report.md      # Stage IV report
| |-- superset_instructions.md  # Superset setup guide
|-- base/
| |-- packages.tex             # LaTeX packages
| |-- style.tex                # LaTeX style
|-- sections/
| |-- *.tex                    # Report sections
|-- report.tex                 # Main LaTeX document
|-- bibliography.bib           # References
```

A.2 Cluster Access Information

- **Cluster:** IU Hadoop cluster (hadoop-01.uni.innopolis.ru)
- **User:** team2
- **HiveServer2:** hadoop-03.uni.innopolis.ru:10001
- **PostgreSQL:** hadoop-04.uni.innopolis.ru:5432
- **Superset:** http://hadoop-03.uni.innopolis.ru:8808
- **Main script:** /home/team2/project/main.sh

A.3 Sqoop Command

Code extraction 3: Sqoop import command

```
1 sqoop import \  
2   --connect jdbc:postgresql://hadoop-04.uni.innopolis.ru  
   :5432/team2_projectdb \  
3   --username team2 --password V2P1hy6zjPqWoXMm \  
4   --table crimes \  
5   --split-by id \  
6   --num-mappers 4 \  
7   --as-parquetfile \  
8   --compress --compression-codec org.apache.hadoop.io.  
   compress.SnappyCodec \  
   --
```

```
9  --target-dir /user/team2/project/warehouse/crimes
```

A.4 Spark Submit Command

Code extraction 4: Spark ML pipeline submission

```
1  spark-submit \  
2      --master yarn \  
3      --deploy-mode cluster \  
4      --num-executors 4 \  
5      --executor-memory 4g \  
6      --executor-cores 2 \  
7      --driver-memory 4g \  
8  scripts/ml_pipeline.py
```

A.5 Dashboard Screenshots

A.5.1 Stage IV Superset Dashboard

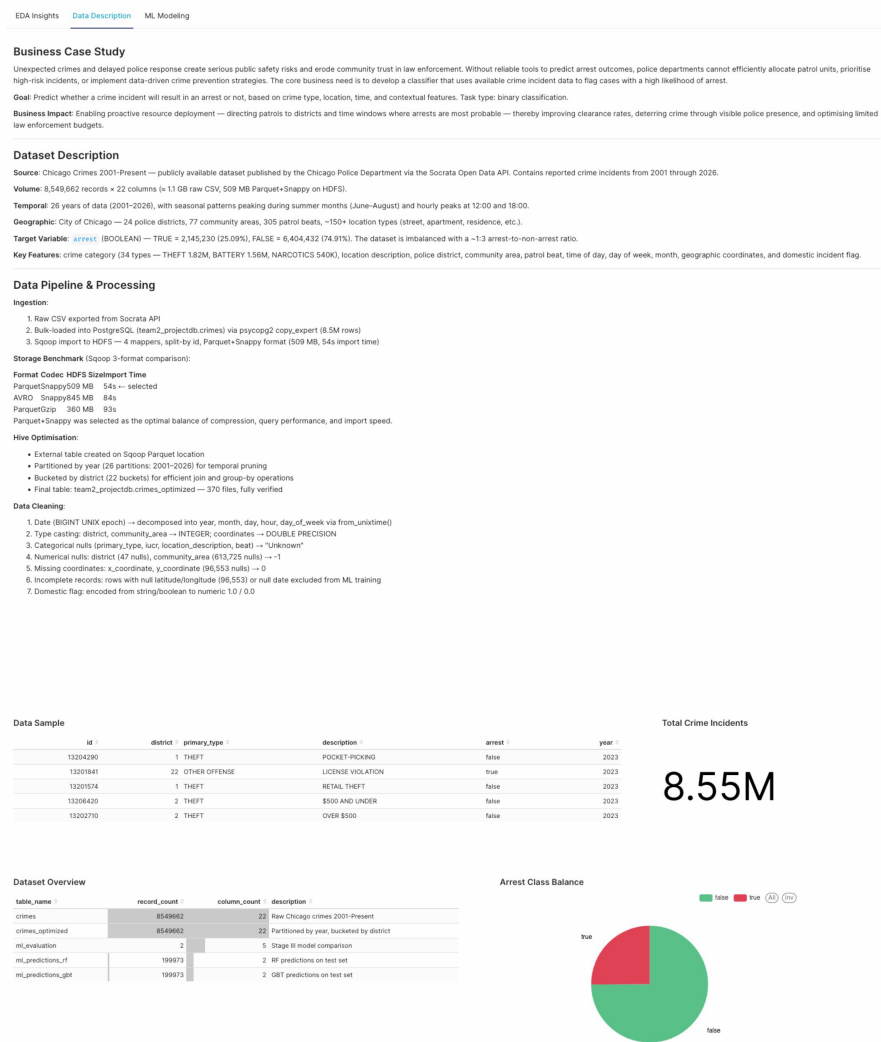


Figure 4: Dashboard Tab 1: Data Description — dataset overview, schema, class balance, and data cleaning summary



Figure 5: Dashboard Tab 1: Data Description (continued) — year range and data cleaning text



Figure 6: Dashboard Tab 2: EDA Insights — arrest rate by district, top crime types, crimes by hour, arrest rate trend, crimes by month, arrest by community area

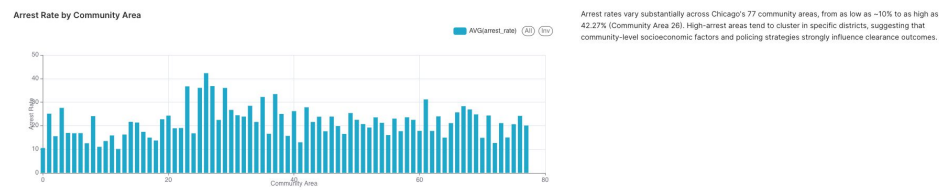


Figure 7: Dashboard Tab 2: EDA Insights (continued) — domestic vs non-domestic arrest rates

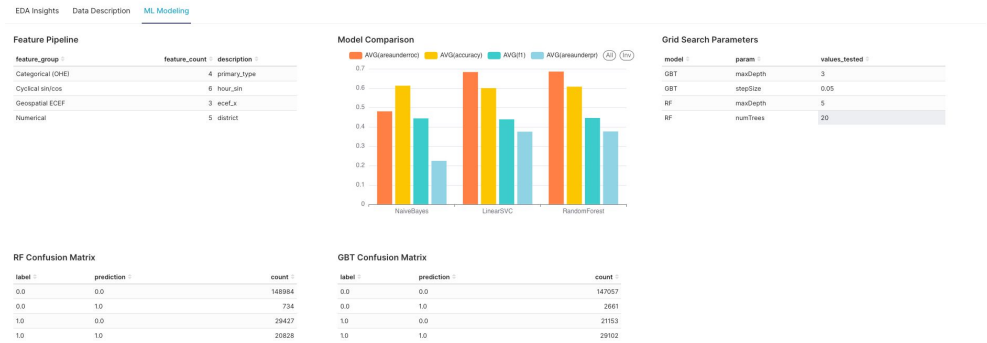


Figure 8: Dashboard Tab 3: ML Modeling — feature pipeline, grid search parameters, model comparison, and confusion matrices

A.5.2 Stage II EDA Dashboard

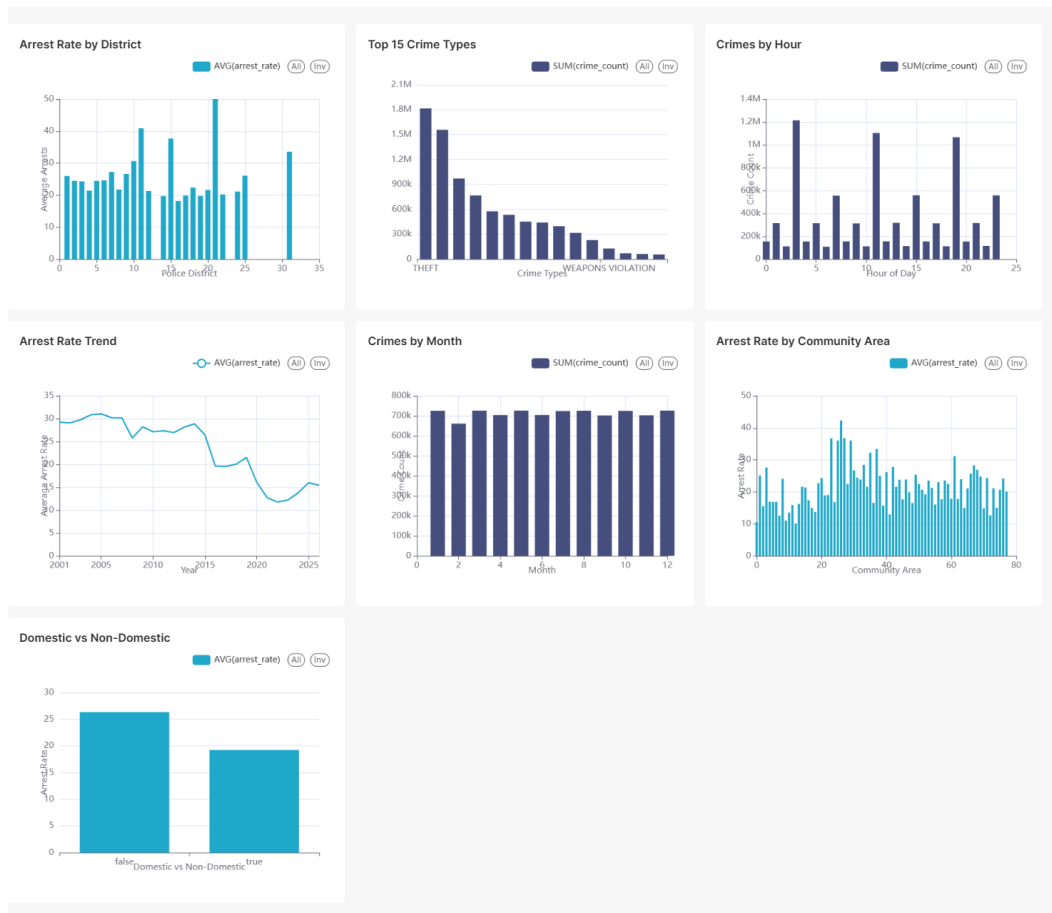


Figure 9: Stage II EDA Dashboard (initial 7 charts)